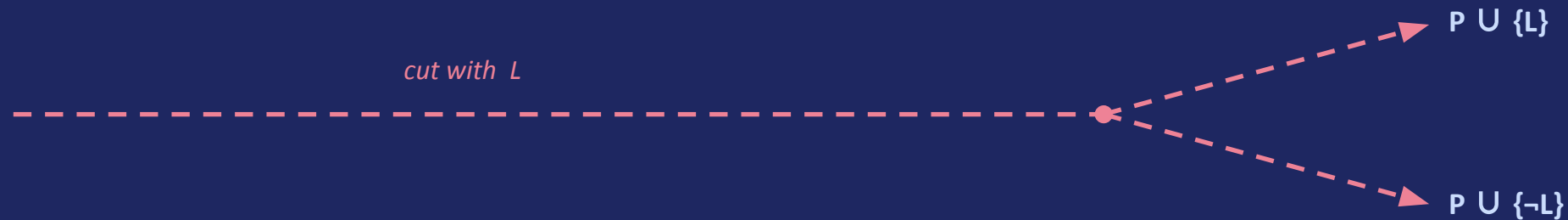


Neural Conjecturing for Saturation Theorem Provers

Learning cut formulas that restructure first-order proof search — verified by the prover itself



Karel Chvalovský¹ **Martin Suda¹** **Josef Urban^{2,3}**

¹ Czech Technical University in Prague ² AI4REASON ³ University of Gothenburg / Chalmers University

Conjecturing: the other half of mathematical skill

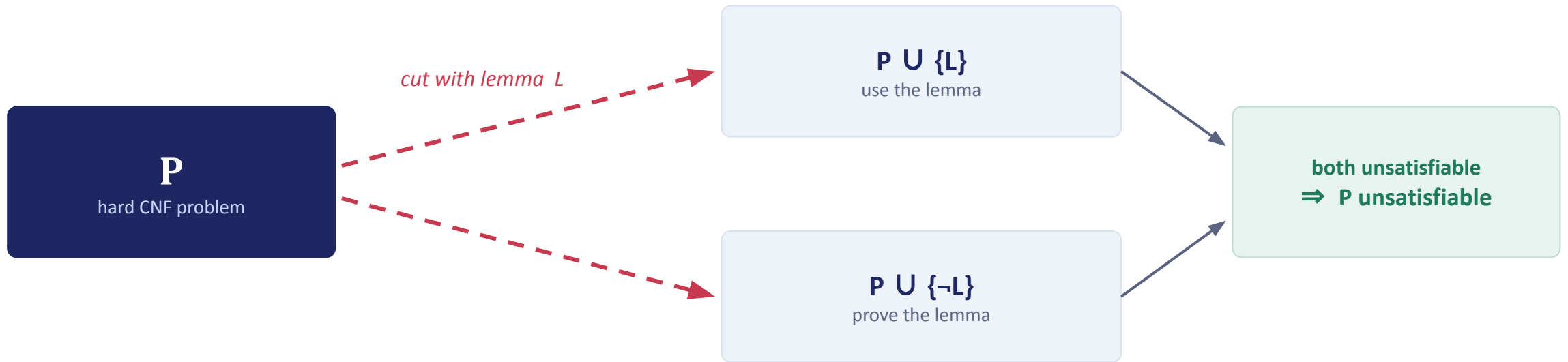
- Mathematicians decompose hard problems into intermediate lemmas — proposing the right lemma is often what separates a feasible proof from an intractable search.
- Saturation provers such as E and Vampire combine many strong heuristics, yet one wrong trajectory can cost millions of inferences before a short refutation.
- Automated conjecturing is an old dream: theory formation by heuristics and controlled enumeration, later neural sequence models over large formal libraries.
- This work: generate intermediate lemmas with a graph neural network — and let the ATP itself decide which ones are useful.



SIX DECADES OF CONJECTURING

- | | |
|-------------|---|
| 1977 | AM (Lenat) — automated theory formation |
| 1988 | Graffiti (Fajtlowicz) — conjectures in graph theory |
| 2002 | HR (Colton) — theory formation in pure mathematics |
| 2020 | First neural conjecturing datasets over Mizar (Urban & Jakubův) |
| 2026 | This work — graph-to-clause cut generation with cross-prover transfer |

Conjecturing for saturation provers: the classical cut



No beauty required

L only has to expose a useful case split — it need not be a human-interesting theorem.



Speculation is sound

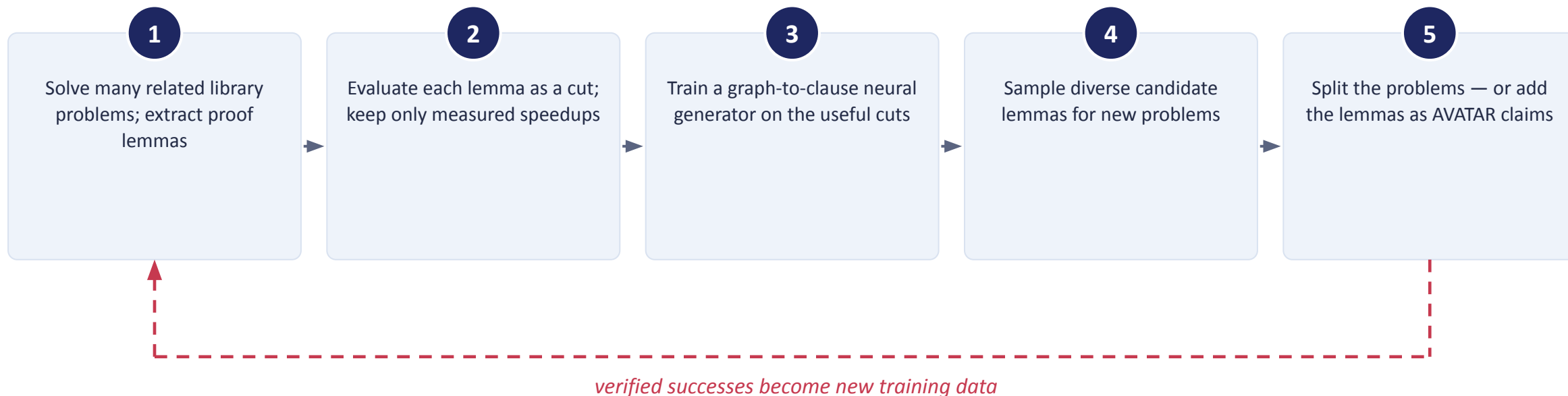
A bad lemma can waste time, but it can never make the prover unsound.



Hints, too

Generated lemmas can also serve as proof-search hints (Veroff 1996).

A prove-learn loop for cut discovery



Supervision by measurement — useful cuts are labeled by the ATP's own search reduction — no human annotation anywhere in the loop.



Developed in the open — github.com/JUrban/E_conj_dev1 · original dataset & code: github.com/JUrban/E_conj

When is a lemma useful? Let the prover measure it

$$r = (C_1 + C_2) / C_0$$

C_0 — cost of proving P from scratch

C_1, C_2 — costs of proving $P \cup \{L\}$ and $P \cup \{\neg L\}$

L is useful for $P \iff$ **both branches prove** and $r < 1$



Cost metric — processed clauses for E ; burned CPU instructions for Vampire — deterministic and comparable across runs.



No human labels — usefulness is defined purely by measured reduction of ATP proof search.

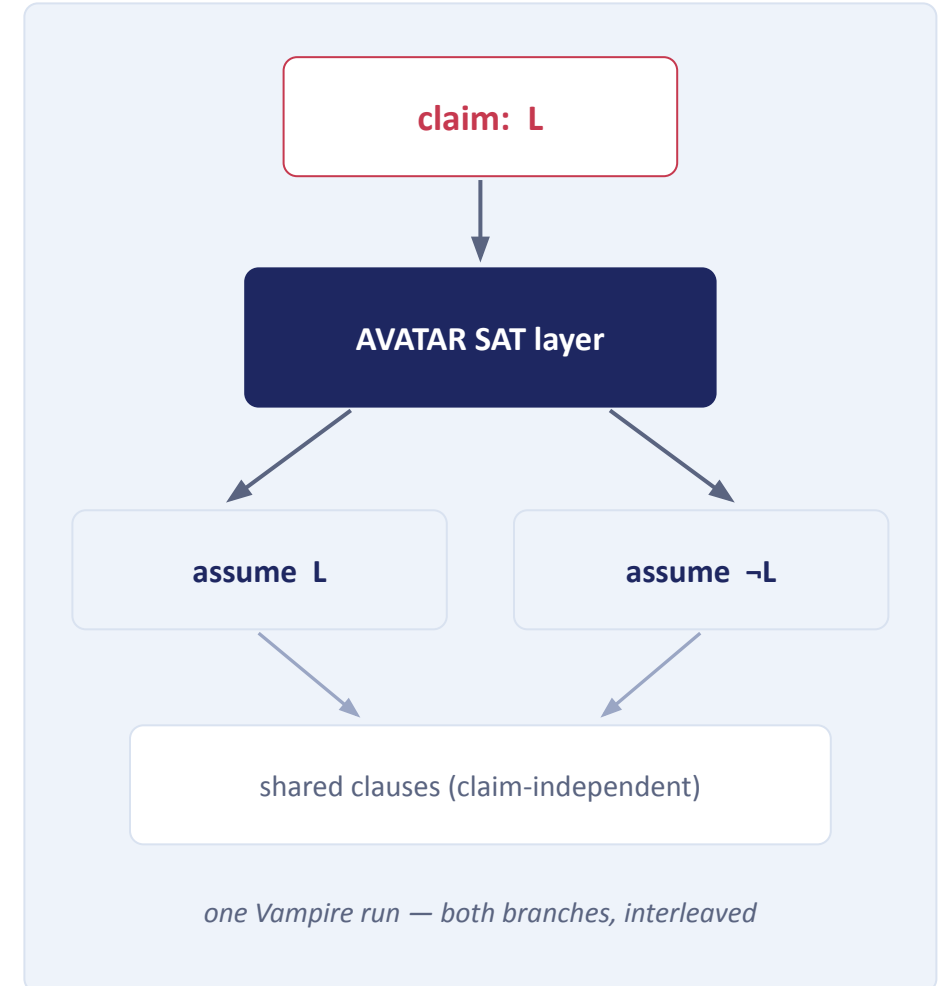


Starting point — a dataset of 3,161 Mizar CNF problems with cut ratios (Goertzel & Urban, AITP 2019).

The same signal drives everything downstream: training targets, loss weights, and final evaluation.

AVATAR claims: try many cuts in a single run

- Running two separate proofs per candidate lemma is expensive.
- Vampire's AVATAR architecture can instead take L through the TPTP claim role.
- Its SAT layer creates complementary assumptions for L and $\neg L$, interleaves the two branches, and shares all clauses derived independently of the claim.
- A proof is complete when both sides of the claim are discharged — or when P is proved without touching the claim at all.
- Several generated claims can be explored simultaneously — we later give Vampire up to 42 at once.



Two corpora, one supervision signal

Corpus	Problems	Solved	Evaluated (problem, lemma) pairs	Useful cuts
E · post-proof-minimized Mizar CNF	3,161	1,888	122,356	44,895
Vampire · full-axiom Mizar40 CNF	57,880	34,182	2,470,292	196,199

2.47 M

evaluated (problem, lemma)
pairs in the Vampire corpus

196,199

measured-useful cuts
available for training

3,591

problems with at least
one useful lemma

Useful = both cut branches prove and $r < 1$. Lemmas come from minimized E proofs and from Deepire-guided Vampire proofs. Final evaluation also uses the premise-selected Mizar60 corpus: 55,832 problems, median 34 clauses after clausification — a realistic middle ground between minimized and full-axiom problems.

Deepire 2.0: the neural Vampire underneath



Extraction model (no AVATAR)

Strategy found by hill-climbing on 3,000 Mizar40 problems (3,000 Mi), then 18 iterations of GNN clause-selection training on all 57,880 problems at 16,000 Mi.

40.5% → 57.7% solved



Evaluation model (AVATAR on)

Trained on a random half of Mizar40 (20 iterations, 42.1% → 62.0%), then on premise-selected Mizar60: 22 iterations over 27,916 problems.

60.1% → 70.6% solved



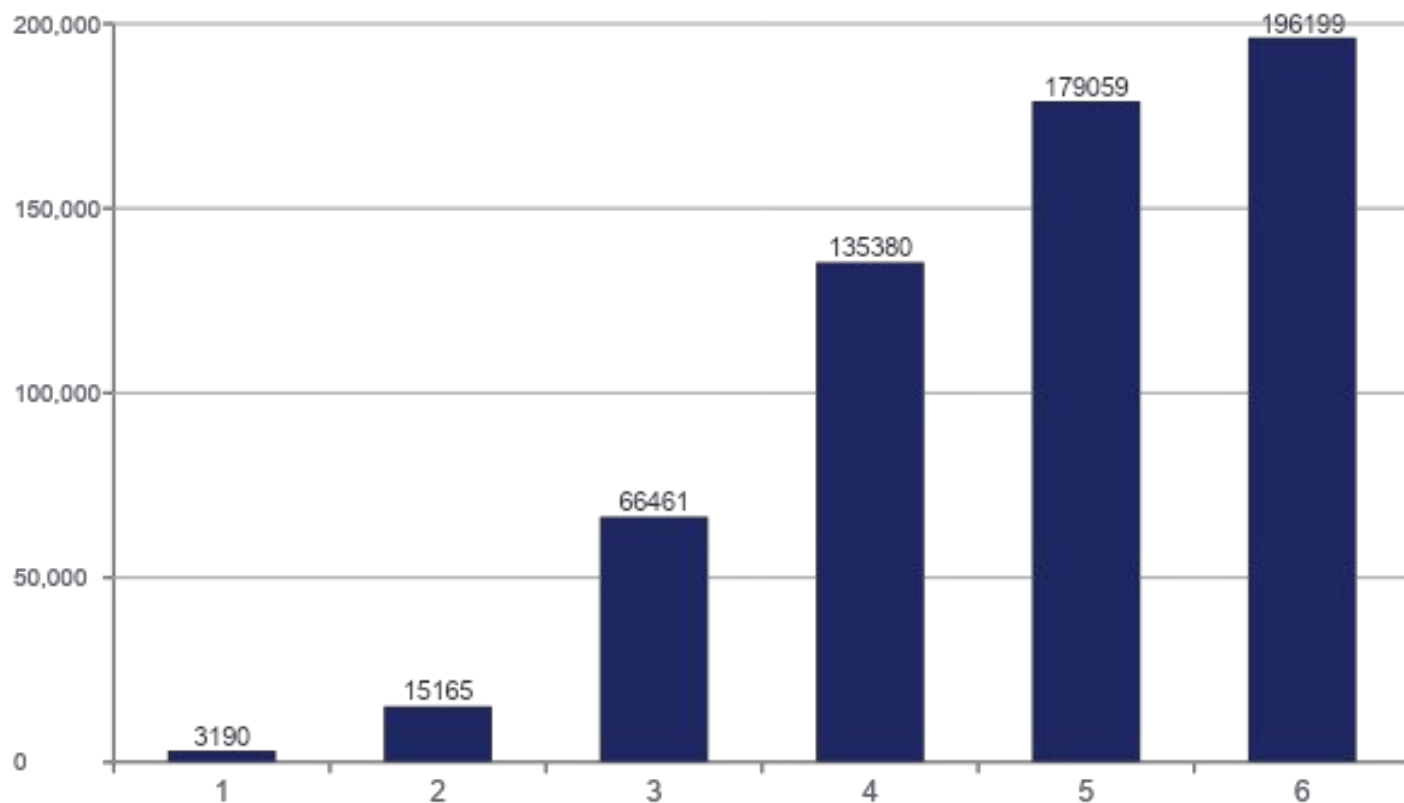
Why extract without AVATAR?

Under AVATAR, derived clauses depend on propositional splits and are equivalent only to more complex formulas — so lemma extraction stays simple on a non-AVATAR run.

620

hard Mizar60 problems — never solved by any ATP in the MizAR 60 evaluation — **already fall to the trained strategy alone**, before any conjecturing. This is the baseline the conjectures must beat.

Useful cuts are heavily skewed toward small gains



Cumulative counts of the 196,199 useful Vampire cut examples by ratio threshold

3,190

examples with $\geq 10\times$ speedup
(on 164 problems)

66,461

examples with $\geq 2\times$ speedup
(on 1,645 problems)



All useful examples are kept for training; loss weighting $1/(1+r)$ makes the high-speedup tail more influential.

The problem as a heterogeneous graph

5 node types

clause

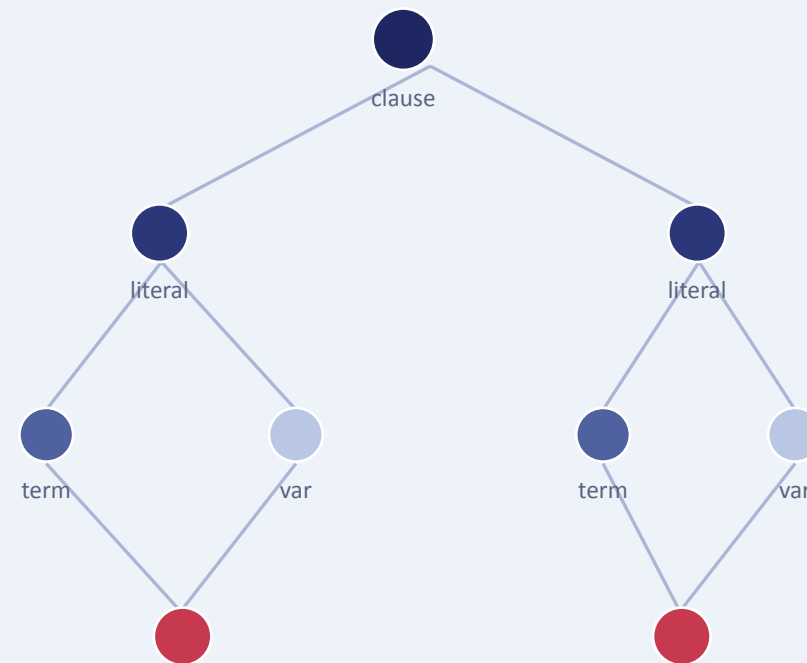
literal

term

variable

symbol

- 16 directed edge types encode containment and syntax: clause $\rightarrow$ literal, literal $\rightarrow$ argument, term $\rightarrow$ subterm — plus reverse edges.
- 4 message-passing layers: edge-aware GINE convolutions on positional edges; GraphSAGE-style aggregation on the rest.
- Output: the symbol-node embeddings — both the memory the decoder attends over and the objects it can point to.



symbol nodes — pointer targets

containment + syntax edges (16 types, directed)

Point at symbols — don't name them

- No fixed vocabulary: predicates and functions are chosen by a pointer distribution over the symbol nodes of the current problem.
- The model relies on arity, role and occurrence structure rather than names — so it applies to symbols never seen in training.
- This anonymous setting was introduced for ATP by Olšák et al. (ECAI 2020); term synthesis in the spirit of Piepenbrock et al. (JSC 2025).
- Optional named embeddings are added for recurring Mizar symbols — unknown symbols (e.g., fresh Skolems) remain generatable from their graph embeddings alone.



Why it matters

New premise selections, other clausifiers, and autogenerated Skolem symbols all change the signature at test time.

A name-based generator breaks. A pointer survives — this is the property behind the cross-prover transfer to Vampire.

Same architecture, any signature — the input problem defines the output vocabulary.

Autoregressive decoding under hard syntax constraints

A clause is generated as a sequence of actions:

NewLitPos

NewLitNeg

Pred

ArgVar

ArgFunc

EndArgs

EndClause

- Each step predicts an action, a symbol pointer when a predicate or function is needed, and a variable slot when a variable is needed.
- Role masks keep predicates and functions apart; an arity stack records how many argument positions remain open.
- Actions inconsistent with the stack are masked to $-\infty$ before sampling — a hard state machine, not a soft preference.

98.9% / 99.3%

syntactic validity at generation scale — across 517,435 first-iteration and 235,089 second-iteration lemmas. The payoff of constrained decoding, compared to earlier unconstrained generation.

One loss, three decoder families



Loss — summed cross-entropies for the action, symbol-pointer and variable heads — irrelevant heads are ignored at each step.



Speedup weighting — each sample is weighted by $1/(1+r)$, so stronger measured speedups matter more.



Per-problem normalization — prevents a few lemma-rich theorems from dominating the objective.



Generation — nucleus sampling under the syntax masks; duplicate clauses are removed before any ATP time is spent.

DECODER VARIANTS

Transformer

the workhorse — $d = 256 / 512 / 1024$

Conditional VAE

a latent variable for diversity

Selective state-space (SSM)

Mamba-style linear-time decoding

Also varied: named Mizar embeddings on/off, sampling temperature and nucleus parameters — 15 configurations in total.

Making 100 K-sample training runs practical



Precompute & cache

Graph construction is a bottleneck — samples are precomputed once and cached in a compact tensor format.



Node-budget batching

The hetero-GNN runs one convolution per edge type, so batches are packed by total node count: 100+ small problems, or just a few huge ones.



Packing-invariant loss

Normalizing the loss by token count makes the objective independent of how samples are packed into batches.



Keep the GPUs fed

Persistent data-loader workers and mixed precision keep the A100s saturated (Leonardo HPC cluster).

The generator proposes — the prover disposes

- Every reported success is prover-verified: a lemma counts only when the corresponding ATP proof succeeds.
- E: each candidate is judged by two independent 2 s runs, on $P \cup \{L\}$ and on $P \cup \{\neg L\}$; the baseline is 10 s without lemmas.
- Vampire: candidates are delivered as AVATAR claims under Deepire 2.0 guidance — 100,000 Mi instruction limit plus a 30 s wall clock.
- Discarded before proving: invalid syntax, wrong symbol roles, arity violations, duplicates modulo printing.
- Baselines use the same prover and limits. Solved baselines report speedup ratios; unsolved baselines report newly proved problems; all counts are per problem, not per lemma.



Two budgets, two purposes

Short E limits → fast, cheap comparison of architecture choices.

Generous Vampire limits → fathom what the approach can reach on hard problems.

Ensembles help two thirds of E’s proved problems

279 test problems · 187 solved by the 10 s baseline · 15 trained configurations · 40–80 sampled lemmas per problem

Decoder	d	Sampling	Helped	%	AvgR ↓
Transformer	256	T = 1.5, k = 25, p = .98 (wide)	96	51.3	0.558
Transformer	256	T = 1.2, k = 15, p = .95 (diverse)	91	48.7	0.574
Transformer	256	T = 1.2, k = 15, p = .95 (balanced)	87	46.5	0.524
Transformer	512	T = 1.2, k = 15, p = .95 (diverse)	80	42.8	0.536
CVAE	256	T = 1.2, k = 15, p = .95	78	41.7	0.597
SSM	256	T = 1.2, k = 15, p = .95	72	38.5	0.612
Transformer	1024	T = 1.2, k = 15, p = .95 (diverse)	67	35.8	0.530
Union of all 15 configurations			127	67.9	0.530

AvgR: best cut ratio averaged over helped problems (lower is better) · all shown configurations use named Mizar symbol embeddings

96 / 187

helped by the best single configuration (51.3%)

127 / 187

helped by the union of all 15 configurations (67.9%)

196×

largest speedup (l101_xreal_1); also 50× and 38× elsewhere

Useful conjectures come early — and from different models

Greedy portfolio of configurations

N	Combination	Helped	Saved
1	Trans-256, wide	96	197 K
2	+ SSM-256	106	218 K
3	+ Trans-256, diverse (A)	113	286 K
4	+ Trans-256, balanced	119	287 K
5	+ Trans-256, diverse (B)	123	292 K
15	<i>oracle union</i>	127	301 K

Problems helped using only the top-k lemmas

k	T-256 wide	T-1024	T-512
1	20	31	35
5	54	53	63
10	70	60	75
20	81	65	—
50	90	67	—

‘Saved’ = baseline processed clauses saved by the best cut on helped problems



Median rank of the first useful conjecture: 2–5. Practical recipe: run a cheap top-k pass across several diverse models, then spend the remaining ATP budget where first successes are complementary — five models already cover 123 of the 127-problem oracle.

Do cut patterns learned from E transfer to Vampire?

- 1 **E-trained generator** — $d = 256$, named Mizar symbol embeddings — trained only on E proof lemmas
- 2 **Generate at scale** — ≤ 50 lemmas for each of 11,881 hard or previously unsolved Mizar60 problems
- 3 **517,435 lemmas** — 98.9% syntactically valid — duplicates and invalid samples discarded
- 4 **Top 42 as AVATAR claims** — all delivered to the same search, each tried speculatively and safely
- 5 **Deepire-guided Vampire** — AVATAR-trained clause guidance · 100,000 Mi + 30 s per problem



Nothing matches at test time

Trained on: E proof lemmas from post-proof-minimized problems.

Tested on: premise-selected problems — different clauses, different guidance, a different proof search, and a different prover.

E-trained cuts nearly double Vampire’s hard solves

Hard / unsolved Mizar60 problems evaluated	11,881
Generated lemmas (98.9% valid)	517,435
Baseline proved by Vampire, no lemmas	3,738
of which hard — never solved by any ATP	620
Previously unproved, now solved with lemmas	582
of which hard	442
Total proved (baseline + lemmas)	4,320
total hard problems proved	1,062
Already-solved problems sped up $\geq 10\%$	2,236
Mean ratio on helped solved problems · median useful rank	0.610 · 3

620 → 1,062

hard Mizar problems solved —
nearly doubled by the lemmas

+582

problems newly proved
(a 15.6% relative increase)

2,236

already-solved problems
sped up by at least 10%

Why does it transfer?



Structural templates

The model appears to learn the shapes of useful case splits rather than prover-specific syntactic artifacts — the effect concentrates on the hardest problems (442 of the 582 new solves).



Symbol independence

Pointer decoding is robust to new Mizar symbols, new premise selections, and the many autogenerated Skolem symbols introduced during clasification.



Speculative but sound

AVATAR claims let Vampire benefit from a conjecture while simultaneously checking it — a natural interface between learning and proving.

A fixed-vocabulary generator would be far less robust to any of these shifts.

Closing the loop: learning from its own verified cuts



New data — the successful Vampire runs yield 24,025 additional useful cut examples.



How to retrain — from scratch overfits; initializing from the E checkpoint is better — the best validation loss mixes the E and Vampire cuts (59 K combined examples).



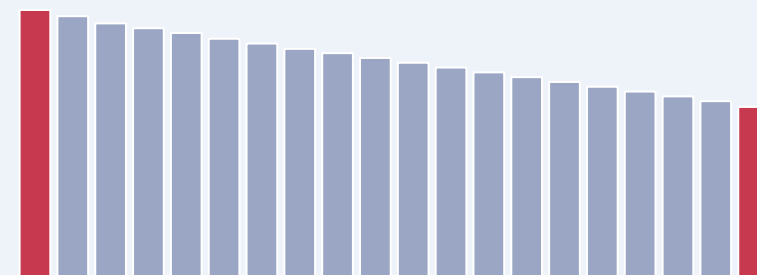
Second-iteration model — generates 235,089 conjectures (≤ 20 per problem) at 99.3% syntactic validity.



Result — +59 newly solved hard problems beyond iteration 1 $\rightarrow$ 1,121 cumulative, an 81% increase over the 620-problem baseline.

Diversity pays to the last rank

In the second iteration, every conjecture rank from 1 to 20 independently solves 356–558 hard problems — even rank 20 still yields 356 (the no-conjecture baseline solves 620).



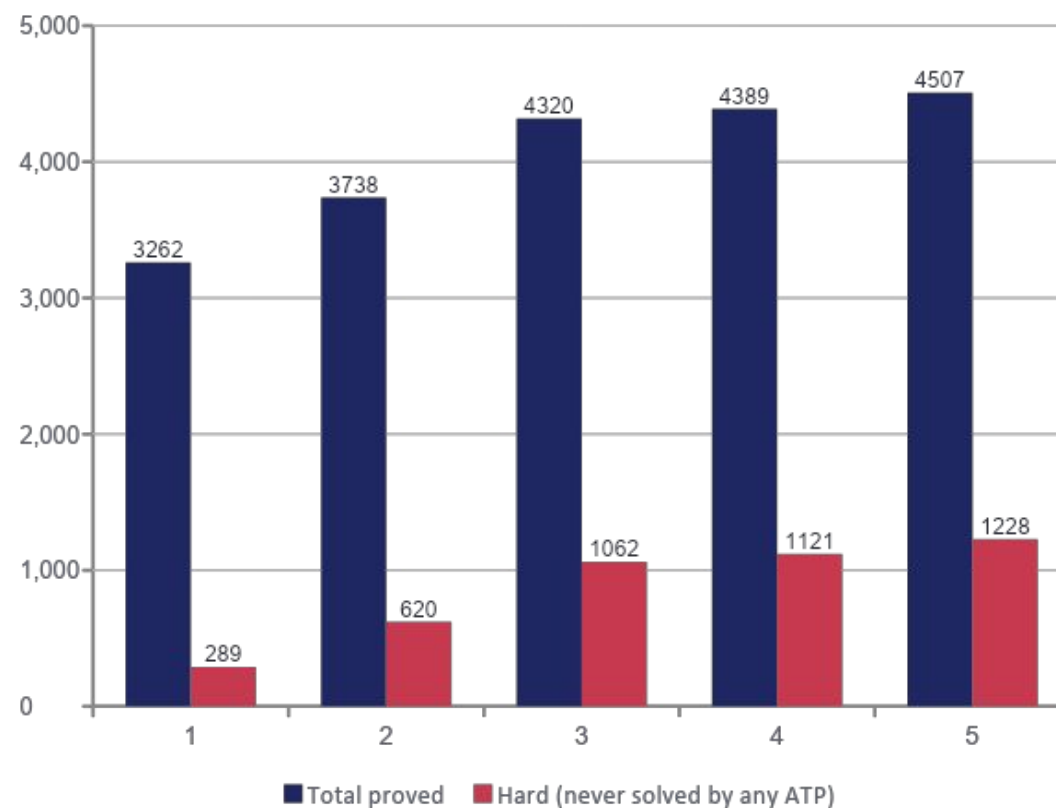
hard problems solved per rank, 1 $\rightarrow$ 20 (schematic; 558 down to 356 — cf. Fig. 1)

Roughly doubling the hard-problem baseline

Configuration	Total	Hard
Vampire CASC portfolio (30 s)	3,262	289
Deepire AVATAR + neural, no conjectures	3,738	620
+ 1st-iteration conjectures (42, E-only)	4,320	1,062
+ 2nd-iteration conjectures (20, mixed)	4,389	1,121
+ CASC portfolio (full union)	4,507	1,228

11,881 hard / unsolved Mizar60 problems · 'Hard' = never solved by any ATP in the MizAR 60 evaluation

One trained strategy beats the whole portfolio on hard problems (620 vs 289). Conjecturing lifts it to 1,121; the full union reaches 1,228 — about twice the baseline.



What we learned — and what could bite



Three practical lessons

1

Judge conjectures by downstream proof search, not language-model likelihood — some low-probability samples create the largest speedups.

2

Diversity is essential — across architectures, dimensions and sampling regimes; useful claims occur at many list positions.

3

Speculative proving is the natural interface — claims let the prover use a conjecture while checking it.



Threats to validity

1

Cut evaluation is expensive, and a deployment pays for every failed candidate — rank scheduling helps; the ATP remains the final ranker.

2

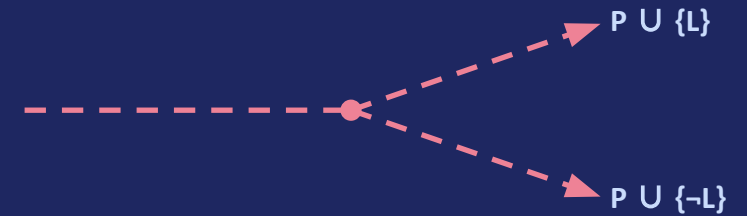
The datasets reflect the prover strategies that produced them — generality needs fresh strategies and more libraries.

3

A learned utility model could pre-filter candidates — but the final evaluation should stay downstream proof search.

Speculate freely — verify soundly

- Neural cut generation works when the prover is the judge: a lemma counts only if the ATP proves both sides of the cut.
- On E, ensembles of generators help 67.9% of baseline-proved problems, with speedups up to 196×.
- Cuts transfer across provers: hard Mizar solves go $620 \rightarrow 1,062 \rightarrow 1,121$; the full union with CASC reaches 1,228 — about twice the baseline.
- Conjecture–prove–learn loops keep improving — the second iteration beats the first.



Code github.com/JUrban/E_conj_dev1

Data github.com/JUrban/E_conj

Thank you — questions welcome